
Distributed Disease Surveillance and Alert System (DSAS)

A Cloud-Native Microservices Platform for
Disease Monitoring, Analytics, and Early Warning

Document Type: Final Year Engineering Project Report
Platform: Cloud-Native Microservices Architecture
Domain: Public Health Informatics / Epidemiological Surveillance
Academic Year: 2024 – 2025
Version: 1.0 (Final Submission)

Abstract

The Distributed Disease Surveillance and Alert System (DSAS) is a cloud-native microservices platform designed to support public health authorities in continuous disease monitoring, epidemiological analytics, and automated early-warning alert generation. Built on an event-driven microservices architecture, it is containerised with Docker, orchestrated via Kubernetes, and monitored through Prometheus and Grafana. Asynchronous messaging is handled by RabbitMQ, persistent storage by PostgreSQL, and delivery pipelines by GitHub Actions.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Project Objectives	1
2	System Architecture	2
2.1	Architectural Paradigm	2
2.2	High-Level Architecture Diagram	3
2.3	Technology Stack	4
2.4	Communication Patterns	4
3	Microservices Design	5
3.1	Services Overview	5
4	Event-Driven Architecture and Messaging	7
4.1	Overview	7
4.2	Benefits of Event-Driven Design	7
5	Database, Containerisation, and Orchestration	9
5.1	Database Architecture	9
5.2	Docker Containerisation	10
5.3	Kubernetes Orchestration	10
6	Monitoring, Security, and CI/CD	11
6.1	Observability Stack	11
6.2	Security Architecture	11
6.3	CI/CD Pipeline	12
7	Scalability, Reliability, and Engineering Decisions	13
7.1	Scalability and Reliability	13
7.2	Key Engineering Decisions	14

8 Conclusion and Future Improvements	15
8.1 Summary	15
8.2 Future Improvements	15
A API Endpoint Reference	17
B Service Port Allocation	18
References	19

List of Tables

2.1	DSAS Technology Stack Summary	4
3.1	Microservice Responsibility Summary	5
5.1	Database Partitioning by Service	9
7.1	DSAS Reliability Targets	13
A.1	DSAS API Gateway Endpoint Reference	17
B.1	DSAS Default Port Allocation	18

List of Figures

2.1	High-Level DSAS System Architecture (Cloud-Native Microservices)	3
4.1	Disease Case Registration Event Flow	7
6.1	DSAS CI/CD Pipeline Stages	12

Chapter 1

Introduction

1.1 Background and Motivation

Disease surveillance is a cornerstone of modern public health. The COVID-19 pandemic exposed critical deficiencies in global monitoring infrastructure — systems unable to scale under demand, slow analytics pipelines, and delayed alert mechanisms. DSAS is a direct response to these challenges: a production-grade, cloud-native platform providing public health authorities with a unified, scalable environment for disease case management, epidemiological analytics, and automated early-warning alerts.

1.2 Project Objectives

DSAS is designed to achieve the following goals:

- Continuous ingestion and tracking of disease case data from multiple sources.
- Registration, retrieval, and updating of patient records.
- Aggregated epidemiological statistics and trend analysis.
- Asynchronous, event-driven processing of disease-related events.
- Automated outbreak detection and alert generation when thresholds are exceeded.
- Geographic association of disease cases for spatial analysis.
- Scalable and resilient infrastructure via Kubernetes and horizontal scaling.

Chapter 2

System Architecture

2.1 Architectural Paradigm

DSAS is built on a **microservices architecture** — a design approach decomposing the application into small, independently deployable services, each responsible for a distinct bounded context. This enables independent scaling, technology heterogeneity, fault isolation, and long-term maintainability.

2.2 High-Level Architecture Diagram

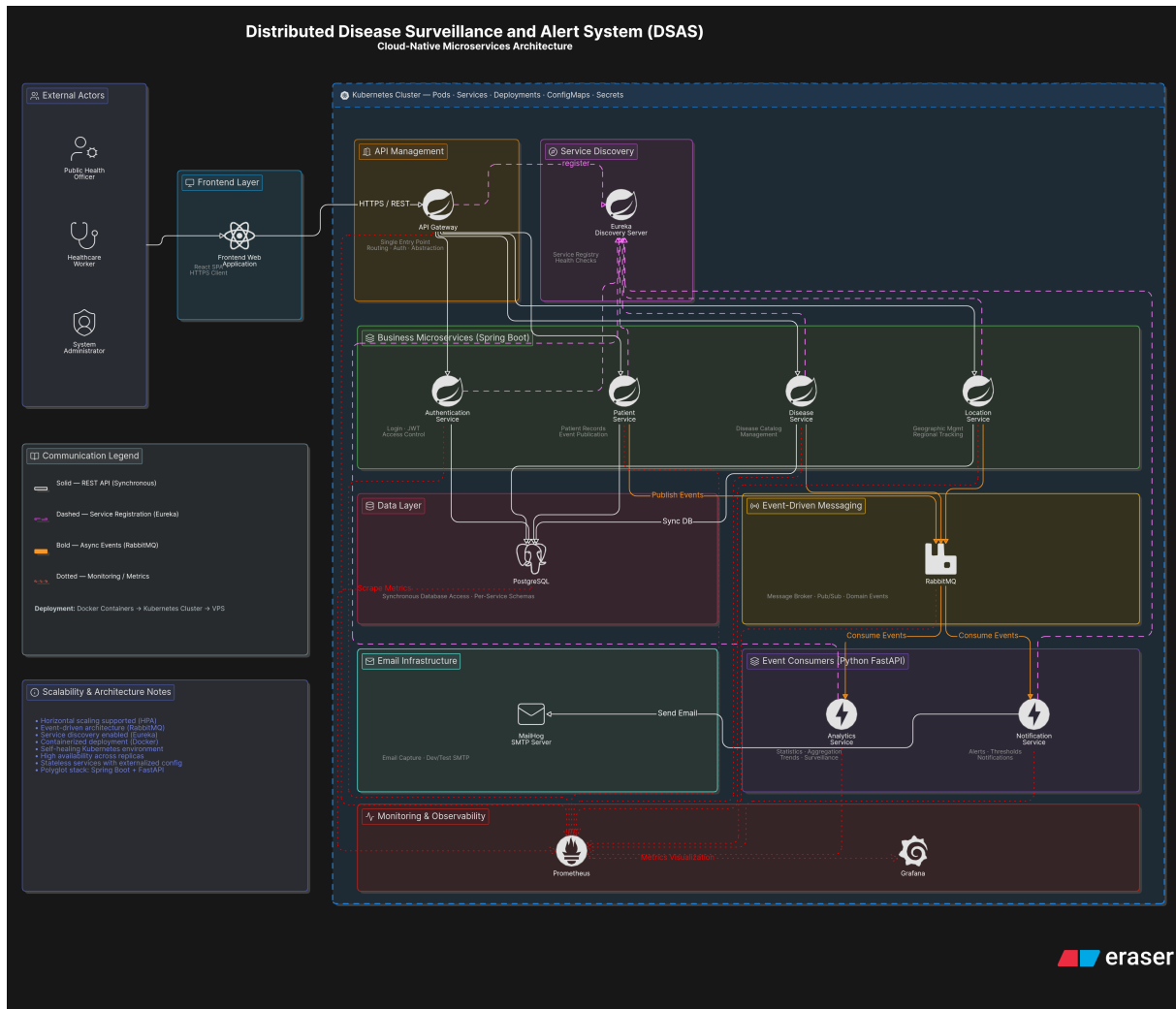


Figure 2.1: High-Level DSAS System Architecture (Cloud-Native Microservices)

2.3 Technology Stack

Table 2.1: *DSAS Technology Stack Summary*

Category	Technology	Role
Backend (Java)	Spring Boot	Core microservice framework
Backend (Python)	FastAPI	Analytics and Notification services
Service Discovery	Netflix Eureka	Dynamic service registry
API Gateway	Spring Cloud GW	Routing, access control, traffic management
Message Broker	RabbitMQ	Asynchronous inter-service messaging
Database	PostgreSQL	Relational persistent storage
Containerisation	Docker	Service packaging and isolation
Orchestration	Kubernetes	Container lifecycle, scaling, networking
Monitoring	Prometheus	Metrics scraping and time-series storage
Dashboarding	Grafana	Metric visualisation and alerting
CI/CD	GitHub Actions	Automated build, test, and delivery
Container Registry	Docker Hub	Container image distribution

2.4 Communication Patterns

Synchronous REST handles all client-initiated requests via the API Gateway. **Asynchronous event-driven messaging** via RabbitMQ handles internal business event propagation between services, decoupling producers from consumers and improving resilience.

Chapter 3

Microservices Design

3.1 Services Overview

Each DSAS microservice adheres to the principles of single responsibility, loose coupling, high cohesion, and independent deployability.

Table 3.1: *Microservice Responsibility Summary*

Service	Technology	Primary Responsibility
API Gateway	Spring Cloud GW	Entry point; routing; access control
Service Discovery	Spring + Eureka	Dynamic service registry
Authentication	Spring Boot	Identity, JWT issuance, access control
Patient	Spring Boot	Patient records, case registration, event publication
Disease	Spring Boot	Disease catalogue and reference data
Location	Spring Boot	Geographic data and spatial services
Analytics	Python / FastAPI	Epidemiological statistics, event consumption
Notification	Python / FastAPI	Alert generation, threshold evaluation

Key Service Highlights

API Gateway — The single entry point for all external traffic. Enforces JWT authentication, role-based access control, and rate limiting before any request reaches a

downstream service.

Authentication Service — Handles user registration and credential validation. Issues signed JSON Web Tokens encoding user identity and role claims, enabling stateless authentication across the platform.

Patient Service — Manages patient records and disease case associations. Acts as the primary event producer, publishing `DiseaseCaseRegisteredEvent` messages to RabbitMQ upon case registration.

Analytics Service — Implemented in Python/FastAPI to leverage superior statistical libraries (Pandas, NumPy). Consumes RabbitMQ events to compute real-time epidemiological statistics.

Notification Service — Also Python/FastAPI. Evaluates incoming case events against configurable outbreak thresholds and dispatches alerts to registered stakeholders when thresholds are exceeded.

Chapter 4

Event-Driven Architecture and Messaging

4.1 Overview

DSAS uses RabbitMQ as its message broker for all internal business event communication. The Patient Service publishes events (e.g., `DiseaseCaseRegisteredEvent`) to RabbitMQ exchanges; the Analytics and Notification services independently consume these events at their own pace.

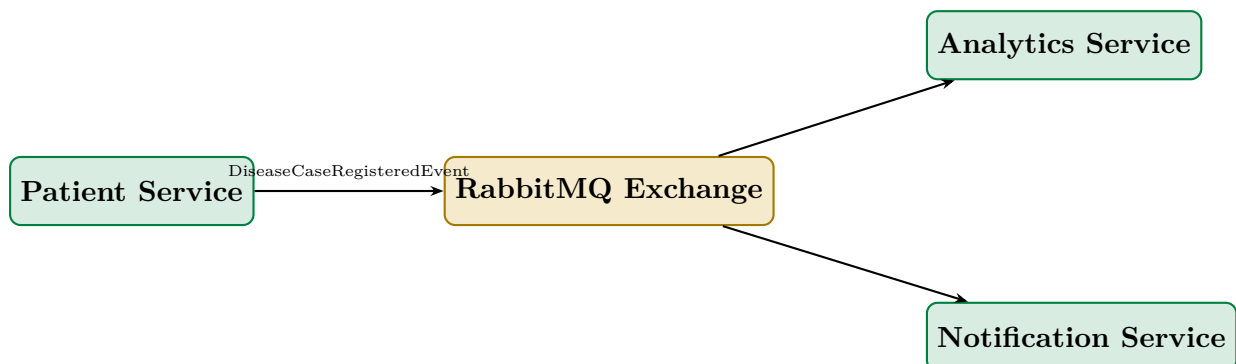


Figure 4.1: *Disease Case Registration Event Flow*

4.2 Benefits of Event-Driven Design

- **Decoupling:** The Patient Service has no knowledge of Analytics or Notification services; it simply publishes events.
- **Resilience:** If a consumer is unavailable, messages queue in RabbitMQ and are processed upon recovery — no data is lost.

- **Scalability:** Consumer replicas scale independently during outbreak surges.
- **Extensibility:** New consumers (e.g., ML inference, reporting) can subscribe to existing event streams without modifying producers.

Chapter 5

Database, Containerisation, and Orchestration

5.1 Database Architecture

DSAS uses **PostgreSQL** with a *Database-per-Service* partitioning strategy: each service has its own isolated schema, enforcing loose coupling at the data layer.

Table 5.1: *Database Partitioning by Service*

Service	Schema	Primary Entities
Authentication	auth_db	Users, credentials, roles
Patient	patient_db	Patients, disease cases, history
Disease	disease_db	Disease catalogue, classifications
Location	location_db	Regions, geographic hierarchies
Analytics	analytics_db	Aggregate statistics, trend data
Notification	notif_db	Alerts, thresholds, notification history

PostgreSQL was selected for its full ACID compliance, relational integrity, and maturity — all essential for sensitive public health data.

5.2 Docker Containerisation

All services are packaged as Docker images, providing process isolation, consistent deployments across environments, and portability across any OCI-compliant infrastructure. Images are built using multi-stage Dockerfiles, run as non-root users, and are tagged with Git commit SHAs for traceability. Final images are published to **Docker Hub**.

5.3 Kubernetes Orchestration

Kubernetes manages the full lifecycle of all DSAS services, providing:

- **Deployments** with rolling updates for zero-downtime releases.
- **Horizontal Pod Autoscaler (HPA)** for dynamic replica scaling based on CPU or custom metrics.
- **Health probes** (liveness, readiness, startup) to detect and recover from service failures automatically.
- **ConfigMaps and Secrets** for injecting configuration and credentials without embedding them in images.
- **Persistent Volumes** to preserve PostgreSQL, RabbitMQ, and Prometheus data across Pod restarts.
- **Namespaces** for logical isolation of platform layers (`dsas-core`, `dsas-analytics`, `dsas-data`, `dsas-monitoring`, `dsas-messaging`).
- **Ingress** (NGINX) for TLS termination and routing all external traffic exclusively to the API Gateway.

Chapter 6

Monitoring, Security, and CI/CD

6.1 Observability Stack

DSAS uses **Prometheus** for metrics scraping and **Grafana** for dashboarding. Spring Boot services expose metrics via `/actuator/prometheus`; Python services use the `prometheus-fastapi` library.

Key metrics include HTTP request rates and latencies, JVM memory usage, RabbitMQ queue depths, database connection pool size, and CPU utilisation per service. Grafana dashboards cover service health, API Gateway performance, analytics processing lag, RabbitMQ monitoring, and Kubernetes infrastructure.

6.2 Security Architecture

Security in DSAS is implemented at multiple layers:

- **JWT Authentication:** The Authentication Service issues HMAC-SHA256 signed tokens. The API Gateway validates all tokens before forwarding requests.
- **Role-Based Access Control (RBAC):** Role claims embedded in JWTs are enforced at the gateway boundary.
- **Secret Management:** All credentials are stored as Kubernetes Secrets — never in source code or images.
- **Network Isolation:** Kubernetes Network Policies restrict inter-service traffic to permitted paths only.
- **TLS:** All external endpoints served over HTTPS, terminated at the NGINX

Ingress Controller.

- **Non-Root Containers:** All containers run as non-privileged users.

6.3 CI/CD Pipeline

The DSAS pipeline runs on **GitHub Actions** and proceeds through: code checkout, static analysis, automated build (Maven/pip), unit and integration testing, Docker image build, Docker Hub push, and optional Kubernetes rolling update trigger. Every deployed image is tagged with its Git commit SHA, creating a full traceability chain from source to production.

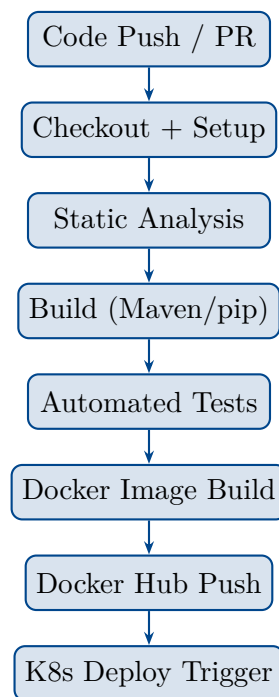


Figure 6.1: *DSAS CI/CD Pipeline Stages*

Chapter 7

Scalability, Reliability, and Engineering Decisions

7.1 Scalability and Reliability

DSAS is designed for horizontal scalability: all business services are stateless, with persistent state held in PostgreSQL or RabbitMQ. During an outbreak surge, the HPA scales service replicas automatically. The RabbitMQ queue buffers event traffic, preventing producer-consumer coupling under load.

Key reliability mechanisms include Kubernetes self-healing (automatic Pod replacement on failure), RabbitMQ durable queues with consumer acknowledgements (preventing message loss), and rolling updates (zero-downtime deployments).

Table 7.1: *DSAS Reliability Targets*

Component	Reliability Mechanism	Target
API Gateway	HPA + Health Probes + ReplicaSet	99.9%
Patient Service	HPA + Health Probes + ReplicaSet	99.9%
Analytics Service	EDA buffer + ReplicaSet	99.5%
Notification Svc	EDA buffer + ReplicaSet	99.5%
PostgreSQL	Persistent Volumes + Backups	99.9%
RabbitMQ	Durable Queues + PVs	99.5%

7.2 Key Engineering Decisions

- **Microservices over monolith:** Enables independent deployability, fault isolation, and technology heterogeneity — essential for a long-lived public health platform.
- **RabbitMQ over Kafka:** Provides sufficient delivery guarantees with lower operational complexity for DSAS scale. Kafka would be preferred at much higher event throughput.
- **PostgreSQL over NoSQL:** ACID compliance and relational integrity are non-negotiable for sensitive patient health data.
- **Kubernetes over Docker Swarm:** HPA, advanced network policies, and rich resource management justify the additional operational complexity.
- **Prometheus + Grafana over commercial tools:** Kubernetes-native, open-source, and cost-free — with no vendor lock-in.

Chapter 8

Conclusion and Future Improvements

8.1 Summary

DSAS successfully realises all project objectives: continuous disease surveillance, near-real-time analytics, automated outbreak detection, geographic tracking, automated alerts, horizontal scalability, and cloud-native deployment. Its event-driven microservices architecture provides resilience, scalability, and maintainability that conventional monolithic surveillance systems cannot match.

8.2 Future Improvements

- **Machine Learning Integration:** Outbreak prediction models (ARIMA, LSTM) and anomaly detection in the Python-based Analytics Service.
- **Multi-Region Deployment:** Geographic redundancy for national or international surveillance programmes.
- **Service Mesh (Istio/Linkerd):** Mutual TLS, distributed tracing, and fine-grained traffic management between services.
- **Spatial Analytics:** Interactive choropleth heatmaps and PostGIS integration for hotspot identification.
- **Enhanced Notifications:** SMS alerts (Twilio/AWS SNS), email digest reports, and WHO/governmental API integration.
- **Data Warehouse:** Long-term epidemiological analytics via ClickHouse or Apache Druid without impacting operational databases.

Closing Statement

The DSAS platform demonstrates that cloud-native engineering principles — microservices decomposition, event-driven communication, container orchestration, and continuous delivery — when applied with architectural discipline, can meaningfully support one of humanity’s most critical endeavours: the protection of public health.

Appendix A

API Endpoint Reference

Table A.1: *DSAS API Gateway Endpoint Reference*

Method	Path	Service	Description
POST	/auth/register	Auth	Register a new user account
POST	/auth/login	Auth	Authenticate and receive JWT
GET	/patients	Patient	List all patient records
POST	/patients	Patient	Register a new patient
GET	/patients/{id}	Patient	Retrieve patient by ID
PUT	/patients/{id}	Patient	Update patient record
POST	/patients/{id}/cases	Patient	Register a disease case
GET	/diseases	Disease	List disease catalogue
POST	/diseases	Disease	Register a new disease
GET	/locations	Location	List geographic regions
GET	/analytics/stats	Analytics	Aggregate disease statistics
GET	/analytics/trends	Analytics	Disease incidence trends
GET	/notifications/alerts	Notification	List generated alerts
POST	/notifications/threshold	Notification	Configure alert threshold

Appendix B

Service Port Allocation

Table B.1: *DSAS Default Port Allocation*

Service	Port
API Gateway	8080
Service Discovery	8761
Authentication	8081
Patient Service	8082
Disease Service	8083
Location Service	8084
Analytics Service	8085
Notification Service	8086
RabbitMQ (AMQP)	5672
RabbitMQ Management	15672
PostgreSQL	5432
Prometheus	9090
Grafana	3000

References

- [1] Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.
- [2] Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- [3] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [4] Hightower, K., Burns, B., & Beda, J. (2019). *Kubernetes: Up and Running* (2nd ed.). O'Reilly Media.
- [5] World Health Organisation. (2006). *Communicable Disease Surveillance and Response Systems*. WHO Press.
- [6] Prometheus Authors. (2024). *Prometheus Documentation*. <https://prometheus.io/docs/>
- [7] RabbitMQ Team. (2024). *RabbitMQ Documentation*. <https://www.rabbitmq.com/documentation.html>
- [8] Fowler, M. (2014). *Microservices*. <https://martinfowler.com/articles/microservices.html>